PART 2 • 80 QUESTIONS

Python Interview Question Answers

Advanced Topics & Core Concepts

Created By : Kartik Mungole

@kartikmungole

Q1 What is the walrus operator (:=)?

The walrus operator (:=), introduced in Python 3.8 (PEP 572), is the assignment expression operator. It assigns a value to a variable as part of a larger expression. Example: while chunk := f.read(8192): process(chunk). It avoids redundant computation by assigning and testing in the same expression.

Q2 What is the 'dataclasses' module?

The dataclasses module (Python 3.7+) provides a @dataclass decorator that auto-generates __init__, __repr__, and __eq__ methods for classes based on annotated fields. It reduces boilerplate for data-holding classes. Optional parameters include frozen=True (immutable), order=True (comparison methods), and field() for default values.

Q3 What are Python type hints?

Type hints (introduced in Python 3.5 via PEP 484) allow you to annotate variables and function signatures with expected types. Example: def greet(name: str) -> str. They are not enforced at runtime but enable static analysis with tools like mypy, improve IDE support, and make code more self-documenting.

Q4 What is async/await in Python?

async/await is Python's syntax for writing asynchronous, non-blocking code using coroutines. 'async def' defines a coroutine function. 'await' suspends the coroutine until the awaited task completes, yielding control to the event loop. Used with asyncio for efficient I/O-bound concurrency without threads.

Q5 What is the 'asyncio' module?

asyncio is Python's standard library for writing concurrent I/O-bound code using an event loop and coroutines. Key functions: asyncio.run(), asyncio.gather() (run coroutines concurrently), asyncio.sleep(), and asyncio.create_task(). It is the foundation for async frameworks like FastAPI and aiohttp.

Q6 What is the 'multiprocessing' module?

The multiprocessing module enables true parallelism by spawning separate processes, each with its own Python interpreter and memory space — bypassing the GIL. Ideal for CPU-bound tasks. Key classes: Process, Pool (for parallel map/apply), Queue, and Pipe for inter-process communication.

Q7 What is the 'threading' module?

The threading module enables concurrent execution using threads within the same process. Threads share memory, requiring synchronization (Lock, RLock, Semaphore, Event) to avoid race conditions. Due to the GIL, threads are best suited for I/O-bound tasks, not CPU-bound parallel computation.

Q8 What is memoization and how is it implemented in Python?

Memoization is an optimization technique that caches the results of expensive function calls so they are not recomputed for the same inputs. In Python, use @functools.lru_cache(maxsize=None) or @functools.cache (Python 3.9+) as a decorator above a recursive or expensive function.

Q9 What is the 'functools' module?

The functools module provides higher-order functions. Key tools: functools.reduce() (applies a function cumulatively), @functools.lru_cache (memoization), functools.partial() (partially applies a function with preset arguments), and functools.wraps() (preserves metadata of wrapped functions in decorators).

Q10 What is the 'itertools' module?

The itertools module provides fast, memory-efficient tools for iterators. Key functions: chain() (chains iterables), cycle() (infinitely cycles), combinations() and permutations() (combinatorics), product() (cartesian product), groupby() (groups consecutive elements), and islice() (slices an iterator without materializing it).

Q11 What is a namedtuple?

A namedtuple is a factory function from the collections module that creates tuple subclasses with named fields. Example: Point = namedtuple('Point', ['x', 'y']); p = Point(3, 4); p.x gives 3. Namedtuples are immutable, memory-efficient, and more readable than plain tuples.

Q12 What is the 'collections' module?

The collections module provides specialized container data types: Counter (counts hashable objects), defaultdict (dict with a default factory for missing keys), OrderedDict (remembers insertion order), deque (double-ended queue with O(1) append/pop from both ends), and namedtuple (tuple with named fields).

Q13 What is the 'heapq' module?

The heapq module implements a min-heap (priority queue) using a regular Python list. Key functions: heapq.heappush(), heapq.heappop() (removes smallest), heapq.heapify() (converts list in-place), heapq.nlargest(), and heapq.nsmallest(). Useful for efficiently finding top-K elements.

Q14 What is multiple inheritance and MRO?

Multiple inheritance allows a class to inherit from more than one parent. The Method Resolution Order (MRO) defines the search order when looking up a method. Python uses the C3 linearization algorithm. Inspect it using ClassName.__mro__ or ClassName.mro(). super() follows the MRO automatically.

Q15 What is a metaclass in Python?

A metaclass is a class whose instances are classes themselves. It controls how a class is created. The default metaclass is 'type'. You define a custom metaclass by inheriting from 'type' and overriding __new__ or __init__. Used in frameworks like Django ORM for declarative class definitions.

Q16 What is __new__ and how does it differ from __init__?

__new__ creates and returns a new instance of a class — called before __init__. __init__ then initializes the already-created instance. __new__ is useful when subclassing immutable types like int or str, or when implementing singleton patterns where you control instance creation directly.

Q17 What is the difference between __getattr__ and __getattribute__?

__getattribute__ is called on every attribute access. __getattr__ is only called when the attribute was not found through the normal mechanism — it is a fallback. Overriding __getattribute__ carelessly can cause infinite recursion; always call super().__getattribute__() inside it.

Q18 What are property decorators?

The @property decorator allows a method to be accessed like an attribute. It enables getter, setter, and deleter functionality with @property, @name.setter, and @name.deleter. This enforces encapsulation by validating or computing values on access without exposing internal implementation details.

Q19 What is a descriptor in Python?

A descriptor is any object that defines __get__, __set__, or __delete__ methods, controlling attribute access on another class. They are the mechanism behind @property, @classmethod, and @staticmethod. Data descriptors define both __get__ and __set__; non-data descriptors define only __get__.

Q20 What is the purpose of __slots__?

__slots__ restricts instance attributes to a fixed set, replacing the default per-instance __dict__. This reduces memory usage significantly for classes with many instances and speeds up attribute access. Instances with __slots__ cannot have arbitrary new attributes added at runtime.

Q21 What is operator overloading in Python?

Operator overloading allows custom classes to define behavior for built-in operators using dunder methods. Examples: __add__ for +, __sub__ for -, __mul__ for *, __lt__ for <, __eq__ for ==, __len__ for len(), __contains__ for 'in'. This allows objects to integrate naturally with Python syntax.

Q22 What is the difference between 'global' and 'nonlocal'?

'global x' inside a function declares that 'x' refers to the module-level global variable. 'nonlocal x' inside a nested function refers to the variable in the nearest enclosing (non-global) scope. Neither keyword is needed just to read a variable — only to reassign it.

Q23 What is exception chaining in Python?

Exception chaining links exceptions using 'raise NewException from original'. The 'from' clause sets the __cause__ attribute. When an exception occurs inside an except block without 'from', Python sets __context__ implicitly. Use 'raise X from None' to suppress the original exception context entirely.

Q24 What is the difference between 'raise' and 'raise from'?

'raise ExceptionType(message)' raises a new exception. Bare 'raise' re-raises the current exception inside an except block. 'raise NewException from original' explicitly chains exceptions, setting __cause__ on the new exception. Use 'raise X from None' to hide the original exception entirely.

Q25 What is string interning in Python?

String interning is an optimization where Python reuses the same object for identical string literals, saving memory. Short strings and identifiers are typically interned automatically. You can manually intern strings using sys.intern(). Never rely on 'is' for string equality checks — always use '=='.

Q26 What are Python's bitwise operators?

Python's bitwise operators work on integers at the binary level: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift). Example: 5 & 3 = 1 (0101 & 0011 = 0001). Used in low-level programming, flags, masking, and performance-critical bit manipulation.

Q27 What is the 'contextlib' module?

The contextlib module provides utilities for context managers. Key tools: @contextlib.contextmanager (turns a generator into a context manager), contextlib.suppress(exceptions) (suppresses specified exceptions), contextlib.nullcontext() (a no-op context manager), and ExitStack (dynamic stacking of context managers).

Q28 What is the difference between 'bytes', 'bytearray', and 'memoryview'?

bytes is an immutable sequence of integers (0–255) for binary data. bytearray is its mutable counterpart. memoryview exposes the buffer protocol of an object, allowing slicing and working with binary data without copying it — critical for performance when working with large binary buffers.

Q29 What is the 'abc' module?

The abc (Abstract Base Classes) module provides infrastructure for defining abstract base classes. Using @abc.abstractmethod, you declare methods subclasses must implement. A class with unimplemented abstract methods cannot be instantiated — raises TypeError. ABCs enforce interface contracts throughout a class hierarchy.

Q30 What is the 'struct' module?

The struct module converts between Python values and C structs represented as bytes. Used for binary data parsing and packing — reading binary file formats or network protocols. struct.pack(format, values) returns bytes; struct.unpack(format, buffer) returns a tuple. Format strings use 'i' (int), 'f' (float), 's' (char[]).

Q31 What is the 'os' module used for?

The os module provides a portable interface to operating system functionality: os.path for path manipulation (join, exists, dirname), os.listdir() to list directory contents, os.getcwd() for current directory, os.makedirs() to create directories, os.environ for environment variables, and os.remove() to delete files.

Q32 What is the 'sys' module used for?

The sys module provides access to interpreter-specific variables and functions: sys.argv (command-line arguments), sys.path (module search path), sys.exit() (exit the interpreter), sys.stdin/stdout/stderr (standard I/O streams), sys.version (Python version string), and sys.getrecursionlimit().

Q33 What is the difference between Python 2 and Python 3?

Python 3 introduced print as a function, true division by default, Unicode strings by default, and removed xrange(). Python 2 reached end-of-life in January 2020. Python 3 also introduced f-strings, type hints, async/await, and many standard library improvements not available in Python 2.

Q34 What is the difference between any() and all()?

any(iterable) returns True if at least one element is truthy; False if the iterable is empty. all(iterable) returns True if all elements are truthy; also True if the iterable is empty. Both use short-circuit evaluation — they stop processing as soon as the result is determined.

Q35 What is the zip() function?

zip() takes multiple iterables and returns an iterator of tuples, pairing elements at the same index. Example: zip([1,2,3], ['a','b','c']) gives (1,'a'), (2,'b'), (3,'c'). It stops at the shortest iterable. Use itertools.zip_longest() to fill missing values with a fillvalue.

Q36 What is the enumerate() function?

enumerate(iterable, start=0) returns an iterator of (index, value) tuples, allowing you to loop with both the index and value simultaneously. Example: for i, val in enumerate(['a','b','c']) gives (0,'a'), (1,'b'), (2,'c'). The 'start' parameter sets the initial counter value.

Q37 What is the re module in Python?

The re module provides regular expression support. Key functions: re.match() (matches at start), re.search() (searches anywhere), re.findall() (returns all matches as a list), re.sub() (replaces matches), re.compile() (compiles a pattern for reuse). Use raw strings (r'\d+') to avoid backslash conflicts.

Q38 What is the difference between re.match() and re.search()?

re.match() only matches at the beginning of the string — if the pattern is not at the start, it returns None. re.search() scans through the entire string for any location where the pattern matches. To match the full string, use re.fullmatch() or add ^ and $ anchors.

Q39 What is JSON and how does Python work with it?

JSON is a lightweight data-interchange format. Python's json module handles serialization and deserialization: json.dumps(obj) converts a Python object to a JSON string; json.loads(string) parses JSON to a Python object. json.dump() and json.load() work with file objects directly.

Q40 What is Python's logging best practice?

Use the logging module instead of print() for production code. Create a logger with logging.getLogger(__name__). Use levels: DEBUG, INFO, WARNING, ERROR, CRITICAL. Configure handlers and formatters to control output. Use lazy formatting: logger.debug('%s', value) instead of logger.debug(f'{value}').

Q41 What is chained comparison in Python?

Python allows chaining comparisons: 1 < x < 10 is evaluated as (1 < x) and (x < 10), where x is evaluated only once. This is more readable and efficient than writing them separately. Chaining works with all comparison operators: ==, !=, <, >, <=, >=.

Q42 What is the difference between list, tuple, and array?

list is a dynamic, mutable sequence holding any type. tuple is immutable and slightly more memory-efficient. array.array (from the array module) stores elements of a single C-type, making it much more memory-efficient for large numeric data. For numerical computing, NumPy arrays are preferred over all three.

Q43 What is the difference between os.path.join and string concatenation?

os.path.join() correctly constructs file paths by inserting the OS-appropriate separator (/ on Unix, \ on Windows) and handling edge cases like trailing slashes. String concatenation is error-prone and platform-dependent. Python 3.4+ pathlib.Path provides an even more readable object-oriented approach.

Q44 What is the difference between str.format() and f-strings?

str.format() uses {} placeholders: '{} {}'.format('hello', 'world'). F-strings (Python 3.6+) embed expressions directly: f'{name.upper()}'. F-strings are faster at runtime and more readable. For Python <3.6, format() is the preferred modern alternative to %-style formatting.

Q45 What is Python's numeric precision issue with floats?

Floating-point numbers follow IEEE 754 standard, which can cause precision issues like 0.1 + 0.2 != 0.3, because many decimal fractions cannot be represented exactly in binary. For precise decimal arithmetic, use the decimal.Decimal class. Always use Decimal instead of float for financial calculations.

Q46 What is a Python virtual environment?

A virtual environment is an isolated Python environment with its own interpreter and packages, separate from the system Python. It prevents dependency conflicts between projects. Created using 'python -m venv env_name', activated with 'source env_name/bin/activate' (Linux/Mac) or 'env_name\Scripts\activate' (Windows).

Q47 What are Python's string methods split(), join(), strip()?

split(sep) splits a string into a list by separator (default: whitespace). join(iterable) joins strings with the calling string as separator: ', '.join(['a','b','c']) gives 'a, b, c'. strip() removes leading and trailing whitespace (or specified characters); lstrip() and rstrip() strip from one side only.

Q48 What is the difference between input() and raw_input()?

In Python 3, input() reads a line from stdin and always returns a string. In Python 2, raw_input() did the same, while input() evaluated the expression (a security risk). Python 3 removed raw_input() entirely and made input() safe by always returning a string that you explicitly convert.

Q49 What is pickling and unpickling?

Pickling serializes a Python object into a byte stream using the pickle module, so it can be saved to a file or transmitted. Unpickling is the reverse — deserializing the byte stream back into a Python object. Use pickle.dump() to pickle and pickle.load() to unpickle. Never unpickle data from untrusted sources.

Q50 What is duck typing?

Duck typing is a concept where the type of an object is less important than the methods or attributes it has. If an object behaves like a duck (has the needed methods), Python treats it as one. This allows for flexible, dynamic code without strict type checking — enabling polymorphism without inheritance.

Q51 What is the difference between a shallow copy and a deep copy?

A shallow copy (copy.copy()) creates a new container but shares references to nested objects. A deep copy (copy.deepcopy()) creates a fully independent object with all nested objects also copied recursively. Assignment (b = a) makes both variables point to the same object — no copy at all.

Q52 What is the difference between positional and keyword arguments?

Positional arguments are passed in order and matched by position. Keyword arguments are passed by explicitly naming the parameter (key=value), allowing them to be in any order. Keyword arguments improve readability and allow skipping optional parameters that have default values.

Q53 What is the difference between 'assert' and raising an exception?

'assert condition, message' raises AssertionError if False. It can be disabled with the -O flag and should never be used for production validation. Raising exceptions (raise ValueError(...)) is the correct approach for runtime error handling that must not be silently bypassed in optimized builds.

Q54 What is the difference between @staticmethod and @classmethod?

@staticmethod defines a method with no implicit first argument — a plain function namespaced inside a class. @classmethod receives 'cls' (the class itself) as the first argument and can access or modify class-level state. @classmethod is commonly used as an alternative constructor pattern.

Q55 What is the difference between mutable and immutable objects?

Mutable objects can be changed after creation — lists, dictionaries, and sets. Immutable objects cannot be changed — integers, floats, strings, tuples, and frozensets. Immutable objects are hashable and can be used as dictionary keys or set elements; mutable objects cannot.

Q56 What are Python modules and packages?

A module is a single Python file (.py) containing functions, classes, and variables. A package is a directory containing multiple modules and an __init__.py file. Modules are imported using 'import module_name' or 'from module import item'. Packages help organize large codebases into namespaces.

Q57 What is the 'with' statement used for?

The 'with' statement is used for context management — ensuring proper acquisition and release of resources. It calls __enter__ at the start and __exit__ at the end, even if an exception occurs. Most commonly used for file handling: 'with open("file.txt") as f:' automatically closes the file.

Q58 What is list comprehension?

List comprehension provides a concise way to create lists. Syntax: [expression for item in iterable if condition]. Example: squares = [x**2 for x in range(10) if x % 2 == 0] creates squares of even numbers. Dictionary and set comprehensions follow the same pattern using {} instead.

Q59 What is a generator expression?

A generator expression is like a list comprehension but uses parentheses: (x**2 for x in range(10)). Unlike list comprehensions, generator expressions produce items lazily on demand, using much less memory. Ideal for large datasets where you don't need all items at once.

Q60 What is the difference between '/' and '//' in Python?

'/' is true division and always returns a float (e.g., 7/2 = 3.5). '//' is floor division and returns the largest integer less than or equal to the result (e.g., 7//2 = 3). For negative numbers, floor division rounds toward negative infinity: -7//2 = -4.

Q61 What is the difference between a set and a frozenset?

A set is a mutable, unordered collection of unique elements defined with {} or set(). A frozenset is its immutable version created with frozenset(). Because frozensets are immutable, they are hashable and can be used as dictionary keys or elements of another set, unlike regular sets.

Q62 What is the difference between 'is None' and '== None'?

Always use 'is None' or 'is not None' rather than '== None'. Since None is a singleton in Python, 'is' checks identity and is guaranteed to work correctly. '== None' can be overridden by a class's __eq__ method, making it unreliable. PEP 8 explicitly recommends using 'is None'.

Q63 What is method resolution order (MRO) and how does super() use it?

MRO is the order in which Python searches classes for a method. super() doesn't just call the parent class — it calls the next class in the MRO. This ensures cooperative multiple inheritance works correctly, with each class called exactly once in a predictable order defined by C3 linearization.

Q64 What is the difference between append(), extend(), and insert() in lists?

append(x) adds a single element to the end. extend(iterable) adds all elements of an iterable to the end. insert(i, x) inserts element x at position i, shifting subsequent elements. append and extend are O(1) amortized; insert is O(n) because it shifts elements to make room.

Q65 What is the difference between map() and a list comprehension?

Both apply a function to each element of an iterable. map(func, iterable) returns a lazy iterator and is slightly faster for simple built-in functions. List comprehensions are more readable, support filtering (if condition), and are generally preferred in Python. Use map() when passing an existing named function directly.

Q66 What is *args and **kwargs in function definitions?

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dictionary. They allow functions to accept variable numbers of arguments. Order must be: regular args, *args, keyword-only args, **kwargs. Used widely in wrappers and decorators.

Q67 What is the difference between 'del', 'remove()', and 'pop()' for lists?

'del list[index]' removes an element at a specific index. 'list.remove(value)' removes the first occurrence of a specific value — raises ValueError if not found. 'list.pop(index)' removes and returns the element at the given index (default: last). pop() from the end is O(1); from middle is O(n).

Q68 What is the difference between recursion and iteration?

Recursion solves a problem by having a function call itself with a smaller sub-problem, using the call stack. Iteration uses loops. Recursion is elegant for tree/graph traversal but risks stack overflow for deep recursion. Iteration is generally more memory-efficient and faster for simple repetition.

Q69 What are Python's built-in functions for sorting?

sorted(iterable, key=None, reverse=False) returns a new sorted list without modifying the original. list.sort() sorts in-place and returns None. Both use Timsort O(n log n). The 'key' parameter accepts a function applied to each element for comparison: sorted(words, key=str.lower).

Q70 What is the purpose of __repr__ vs __str__?

__str__ returns a human-readable string for end users, used by print() and str(). __repr__ returns an unambiguous developer-facing representation, used by the interpreter and repr(). If only __repr__ is defined, it serves as fallback for __str__. Best practice: __repr__ should return a string that could recreate the object.

Q71 What is the difference between a coroutine and a generator?

Generators produce data for iteration using yield. Coroutines are designed for concurrency and can both receive and yield values using 'value = yield'. In modern Python, async def coroutines (used with asyncio) are distinct from generator-based coroutines and are the preferred approach for async programming.

Q72 What is the GIL and when does it matter?

The Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time. It matters for CPU-bound tasks — use multiprocessing to bypass it. For I/O-bound tasks (network, file), threading is effective since threads release the GIL while waiting. Python 3.13 introduces an experimental free-threaded build.

Q73 What is the difference between a class variable and an instance variable?

A class variable is defined directly inside the class (outside methods) and is shared across all instances. An instance variable is defined inside __init__ using self.variable and is unique to each instance. Modifying a class variable via an instance creates a new instance variable that shadows the class variable for that instance only.

Q74 What is PEP 8 and why does it matter?

PEP 8 is Python's official style guide. It covers naming conventions (snake_case for variables/functions, PascalCase for classes), indentation (4 spaces), max line length (79 chars), blank lines, import ordering, and commenting. Following PEP 8 ensures consistent, readable, and professionally written Python code.

Q75 What is the difference between type() and isinstance()?

type(obj) returns the exact class of an object. isinstance(obj, class) checks whether an object is an instance of a class or any of its subclasses. isinstance is preferred for type checking because it properly handles inheritance, while type() does not consider subclasses — making it too strict for most use cases.

Q76 What is monkey patching in Python?

Monkey patching is the dynamic modification of a class or module at runtime, replacing or extending its attributes or methods without changing the original source code. Useful in testing (mocking) and quick fixes, but should be used sparingly as it can make code unpredictable and difficult to maintain.

Q77 What is the difference between __iter__ and __next__?

__iter__ returns the iterator object itself (usually self) and is called when iter() is invoked on an object. __next__ returns the next value and raises StopIteration when exhausted. A class implementing both is a full iterator. An iterable only needs __iter__, which returns a separate iterator object.

Q78 What is the purpose of the 'pass' statement?

'pass' is a no-operation placeholder used when a statement is syntactically required but no action is needed. Common uses: empty function or class bodies during development, empty except blocks when intentionally ignoring an exception, and as a placeholder in abstract method stubs before implementation.

Q79 What is the difference between 'is' and '==' in Python?

'==' checks for value equality — whether two objects have the same value. 'is' checks for identity — whether two variables point to the exact same object in memory. For example: a = [1,2]; b = [1,2]; a == b is True but a is b is False. Use 'is' only for None, True, and False checks.

Q80 What is the difference between 'break', 'continue', and 'pass' in loops?

'break' exits the current loop entirely, skipping any remaining iterations and the else clause. 'continue' skips the rest of the current iteration and moves to the next one. 'pass' does nothing — it is a syntactic placeholder that allows an empty loop body without causing a syntax error.